

DATABASES

Complete Study Guide

Everything for the exam — explained from the ground up

Relational Model · ER Modelling · Normalisation · SQL · Transactions · App Programming

How to use this guide

Read top to bottom — each topic builds on the last. **Blue boxes** = core ideas, **green boxes** = worked examples, **red boxes** = common mistakes, **purple boxes** = step-by-step recipes. Then do the **exercises at the end** and check the **answer key**.

Contents

1	Big Picture: What Is a Database?	3
1.1	ANSI-SPARC: the 3-Level Architecture	3
1.2	ACID (overview — full detail in Transactions)	3
2	Notation & Symbols You'll See	4
2.1	Letters (what each variable usually means)	4
2.2	Symbols (operators & logic)	4
3	The Relational Model	4
3.1	Domains, Schemas, Tuples	4
3.2	Null Values & Three-Valued Logic	5
3.3	Integrity Constraints	5
3.4	Keys	5
3.5	Foreign Keys — links without pointers	7
4	Data Modelling (ER & UML)	7
4.1	The ER Model: 3 ingredients	7
4.2	Attribute types	7
4.3	Relationship sets & cardinalities	8
4.4	Weak entity sets	8
4.5	IS-A: Inheritance (Specialisation & Generalisation)	8
4.6	Aggregation	9
4.7	ER vs. UML class diagrams	9
5	From Conceptual to Relational (Translation Rules)	9
5.1	Eliminating tables	10
5.2	ISA translation — 3 methods	10
5.3	Recursive relations	10
6	Functional Dependencies & Normalisation	10
6.1	Why: the three anomalies	11
6.2	First Normal Form (1NF)	11
6.3	Functional Dependencies (FDs)	11
6.4	Armstrong's Axioms & Closure	12
6.5	Canonical (Minimal) Set of FDs	12
6.6	Finding all minimal keys	12
6.7	The Normal Forms	13
6.8	BCNF decomposition (always lossless, may lose FDs)	13
6.9	3NF synthesis (always lossless <i>and</i> FD-preserving)	14
6.10	Multivalued Dependencies (MVDs) & 4NF	14
6.11	Denormalisation	14
7	SQL — Structured Query Language	14
7.1	Sample data used in the SQL examples	14
7.2	The shape of a query & its logical order	15
7.3	Joins	15
7.4	Self joins	15
7.5	Duplicates	15
7.6	Non-monotonic queries (negation): the hard ones	16
7.7	Subqueries — where they go	16
7.8	Aggregation	16
7.9	GROUP BY and HAVING	16

7.10 Set operations	17
7.11 CASE and COALESCE	17
8 Transactions	17
8.1 What goes wrong without isolation (anomalies)	17
8.2 Formal model: schedules & serializability	18
8.3 Conflicts & conflict serializability	18
8.4 Precedence graph — the fast test	18
8.5 Two-Phase Locking (2PL)	19
8.6 Flavours of 2PL (exam favourites)	19
8.7 Recoverable & cascadeless schedules	19
8.8 Other concurrency control	19
8.9 Multi-granularity locking (intention locks)	20
9 Database Application Programming	20
9.1 Application architectures	20
9.2 Dynamic SQL with JDBC	20
9.3 SQL Injection — the one security rule	20
9.4 Object-Relational Mapping (ORM)	21
9.5 LINQ	21
10 One-Page Cheat Sheet	21
11 Exercises	22
12 Answer Key	24

1 Big Picture: What Is a Database?

The two words you must know

A **Database (DB)** = an organised collection of data with a clear structure, a fixed meaning, and a group of users that share it.

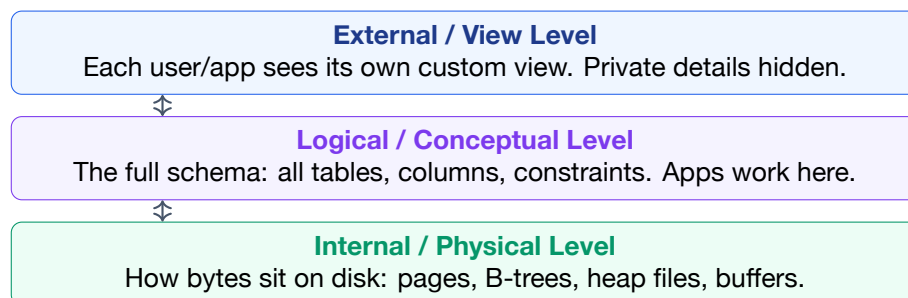
A **Database Management System (DBMS)** = the software that creates, queries, and protects that data (e.g. PostgreSQL, MySQL, SQLite).

Why not just use files or a spreadsheet? Because a DBMS gives you things files cannot:

- **Declarative queries (SQL)** — say *what* you want, not *how* to get it.
- **Fast access** — automatic optimisation, indexes.
- **Data independence** — apps see a logical view; storage is hidden.
- **No duplication** — store each fact once.
- **Integrity constraints** — enforce rules (keys, foreign keys, checks).
- **Concurrent access** — many users at once, no corruption.
- **Transactions (ACID)** — correctness even under crashes.
- **Persistence & security** — survives restarts; access control.

1.1 ANSI-SPARC: the 3-Level Architecture

The big idea: **separate what users see from how data is stored**. Three layers:



Data Independence — why we bother with 3 levels

Logical independence: change the logical schema (add a column) without breaking apps that use views.

Physical independence: change storage (add an index, swap engine) without changing the logical schema.

SQL is *declarative*

You describe the result you want; the DBMS's **query optimiser** figures out the efficient plan. Slogan: **SQL = Logic + (implicit) Control**. Opposite of imperative code where you write the exact algorithm.

1.2 ACID (overview — full detail in Transactions)

Every transaction must keep **ACID**:

- **Atomicity** — all-or-nothing. Whole thing commits, or none of it.
- **Consistency** — DB goes from one valid state to another; all constraints hold after commit.
- **Isolation** — concurrent transactions *look* like they ran one at a time.
- **Durability** — once committed, it survives crashes.

2 Notation & Symbols You'll See

Exams and slides use lots of single letters and symbols. Here is every one, in plain words.

2.1 Letters (what each variable usually means)

Symbol	Means
R, S	A relation (table). $R(A_1, \dots, A_n)$ = table R with those columns.
A_i, B_j	A single attribute (column). The subscript just numbers them.
D	A data type ; $\text{dom}(D)$ = its domain (all possible values).
t, u	Tuples (rows). $t.A_i$ = the value of column A_i in row t .
I	A database state (instance): the actual rows currently in each table.
s	A relation schema (the column list + types), not the data.
α, β, γ	Sets of attributes (used in FDs). $\alpha \rightarrow \beta$ reads “ α determines β ”.
F	A set of functional dependencies .
$T_1, T_2, \dots$	Transactions . $R_i(X) = T_i$ reads item X ; $W_i(X) = T_i$ writes X .

2.2 Symbols (operators & logic)

Symbol	Means
$\rightarrow$	Functional dependency (“determines”) or a foreign-key reference.
$\twoheadrightarrow$	Multivalued dependency (used for 4NF).
α^+	Closure of α : every attribute α can determine.
$\subseteq$ / $\in$	“is a subset of” / “is an element of”.
$\cup$ / $\cap$	set union / intersection .
$\exists$ / $\forall$	“there exists” / “for all”. Trick: $\forall x \varphi \equiv \neg \exists x \neg \varphi$.
$\neg$	not (negation).
$\equiv$	“is equivalent to”.
$T_i \rightarrow T_j$	In a precedence graph : T_i must come before T_j .

3 The Relational Model

This is the math behind tables. Get the vocabulary right and everything else follows.

3.1 Domains, Schemas, Tuples

Core definitions

Data type D has a **domain** $\text{dom}(D)$ = the set of all values it can hold (e.g. $\text{int} \rightarrow$ all integers).

Relation schema = an ordered list of named, typed columns: $s = (A_1:D_1, \dots, A_n:D_n)$, short form $R(A_1, \dots, A_n)$.

Tuple (row) = one value per column: $t = (d_1, \dots, d_n)$ with each $d_i \in \text{dom}(D_i)$. Read a field as $t.A_i$.

Database state I = maps each relation name to a *finite set* of tuples. In the pure model: **no duplicate rows, no row order**.

Common SQL types: `varchar(n)` (strings), `int` / `float` / `numeric(p,s)` (numbers, `p` total digits, `s` after the dot), `date` / `time` / `timestamp`, `blob` (binary). You can even define your own: `CREATE DOMAIN PosSmall AS numeric(2,0) CHECK(value > 0);`

3.2 Null Values & Three-Valued Logic

Null is not zero, not empty string

NULL is a special marker for *missing* / *not applicable*. SQL uses **three-valued logic**: **TRUE**, **FALSE**, **UNKNOWN**. Any comparison with null gives **UNKNOWN**.

So `WHERE A = NULL` *never* matches. Use `WHERE A IS NULL`. Mark columns `NOT NULL` when a value is always required — it removes surprises.

3.3 Integrity Constraints

Rules that say which database states are legal:

- `NOT NULL` — column can't be null.
- `PRIMARY KEY` / `UNIQUE` — each key value appears at most once.
- `FOREIGN KEY (REFERENCES)` — referencing value must match a key in another table.
- `CHECK(predicate)` — values must satisfy it, e.g. `CHECK(age >= 18 AND age <= 150)`.

3.4 Keys

Start from one idea: a **key uniquely identifies a row**. Everything else is just *how big* the set of columns is. Four words, from loosest to strictest:

The key vocabulary — one ladder

1. **Superkey** — *any* set of columns that is unique per row. “Identifies a row, maybe with extra columns along for the ride.” **Any superset of a superkey is also a superkey.**
2. **Candidate key (= minimal key)** — a superkey with *nothing removable*: drop any column and it stops being unique. A table can have *several* candidate keys.
3. **Primary key** — the *one* candidate key you pick as the official identifier. Must be `NOT NULL` and should never change.
4. **Alternate (secondary) key** — every candidate key you did *not* pick as primary.

Containment: `primary key` $\in$ `candidate keys` $\subseteq$ `superkeys`.

Find the keys in one table

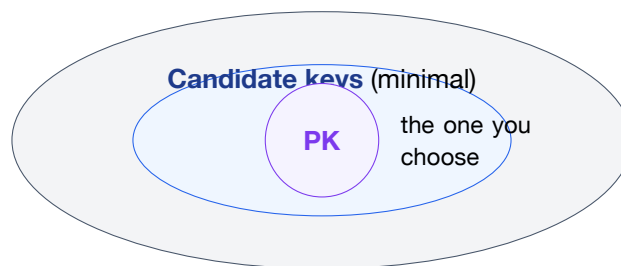
Student(sid, email, name, birthYear). Assume sid is unique and email is unique.

sid	email	name	birthYear
1	ana@x.com	Ana	2003
2	bob@x.com	Bob	2003
3	cleo@x.com	Ana	2004

- **Candidate keys:** {sid} and {email} — each alone identifies a row, and neither can shrink. *Two* candidate keys here.
- **Superkeys:** {sid}, {email}, {sid, name}, {email, birthYear}, {sid, email, name, birthYear}, ... — any set that *contains* a candidate key. {sid, name} is a superkey but *not* candidate, because name is dead weight (sid alone already works).
- **Not a key:** {name} (Ana repeats) and {name, birthYear} (Ana/2003 vs Bob/2003 — still not unique here, and not guaranteed unique in general).
- **Primary key:** pick {sid} (stable synthetic id). Then {email} is an **alternate key** — enforce it with UNIQUE.

Picture the nesting — the smallest circle is what you actually want:

Superkeys (unique, maybe with extra columns)



How to test if a set is a key (link to FDs)

A column set α is a **superkey** of R exactly when $\alpha \rightarrow \text{all attributes}$ — i.e. its closure $\alpha^+ = \text{every column}$. It is a **candidate key** if additionally no proper subset has that property. (Full recipe in the Normalisation section: “Finding all minimal keys”.)

Common mix-ups

- “Primary key” is a *choice*, not a property of the data — the data gives you candidate keys; you promote one.
- A key can be **several columns together** (a *composite* key), e.g. (branchName, deptName). That whole pair is *one* key.
- Adding columns to a key never breaks uniqueness — it just makes a (non-minimal) superkey.
- Only the primary key is forced NOT NULL; alternate keys are enforced with UNIQUE (which *can* allow a null, depending on the DBMS).

Design tip: prefer a single-column, never-updated primary key (e.g. a synthetic id). It keeps foreign keys short and indexes fast. Keep real-world unique fields (like email) as UNIQUE alternate keys.

3.5 Foreign Keys — links without pointers

Foreign key = referential integrity

The relational model has **no pointers**. Instead a **foreign key (FK)** stores the primary key of the row it points to, as a normal column. The FK constraint guarantees every non-null FK value matches a real key in the referenced table.

- Only *keys* can be referenced.
- On delete/update of the target, the DBMS can **REJECT**, **CASCADE**, or **SET NULL**.
- Schema notation: `Results(sid→Students, (category,number)→Exercises, points)`

A tiny schema

`Students(sid, name)` `Exercises(category, number, maxpoints)`
`Results(sid→Students, category, number, points) where (category,number)→Exercises.`
 A `Results` row can only exist for a student and an exercise that actually exist.

Same schema with **actual rows**. The arrows show the foreign keys pointing at the keys they reference:

Students		Exercises			Results			
sid	name	category	number	maxpoints	sid	category	number	points
1	Ana	SQL	1	10	1	SQL	1	8
2	Bob	SQL	2	20	1	SQL	2	15
3	Cleo				2	SQL	1	6

FK in action

A row `Results(9, SQL, 1, 5)` would be **rejected**: there is no student with `sid = 9`. A row `Results(1, BAKING, 1, 5)` is also rejected: no such exercise. That is **referential integrity** — FKs can only point at rows that exist.

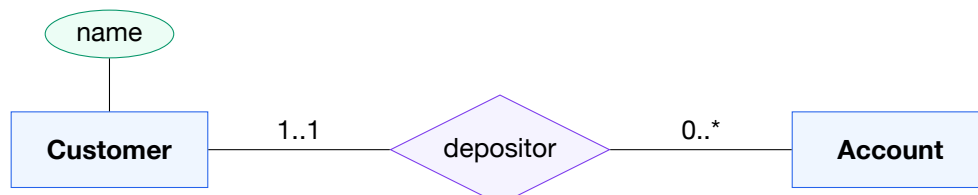
4 Data Modelling (ER & UML)

Before writing tables you design *conceptually*: what info exists and how it connects.

Three design phases

Conceptual (ER / UML — what to store) → **Logical** (turn it into relational tables) → **Physical** (indexes, partitioning, tuning for speed).

4.1 The ER Model: 3 ingredients



- **Entity set** — a collection of similar objects (Customers, Accounts). Drawn as a **rectangle**.
- **Attribute** — a property (name, street). Drawn as an **ellipse**.
- **Relationship set** — an association among entity sets (depositor). Drawn as a **diamond**.

4.2 Attribute types

- **Simple** — atomic (age, id).
- **Multi-valued** — many values (phone numbers). *Double ellipse*.
- **Composite** — has sub-parts (name = first + last).
- **Derived** — computed (age from birthdate). *Dashed ellipse*.

4.3 Relationship sets & cardinalities

A relationship set links entities from $E_1, \dots, E_n$; the **degree** = how many entity sets take part (binary = 2, ternary = 3). Relationships can have **their own attributes** (e.g. salary on actor-plays-in-movie).

Cardinality limits (UML min..max)

On each side you write how many entities of the *other* side one entity connects to.

- 0..* — 0..*: **many-to-many** (default).
- 1..1 — 0..*: **one-to-many** (each B belongs to exactly one A).
- 1..1 — 1..1: **one-to-one** (bijective).
- 0..1 — 0..1: partial function both ways.

Translate words: “exactly one” = 1..1, “at most one” = 0..1, “one or more” = 1..*, “any number” = 0..*.

4.4 Weak entity sets

Weak entity = can't stand alone

A **weak entity set** has *no key of its own*. It is identified by an **identifying entity** through a **total, one-to-many identifying relationship** (drawn as a *double diamond*; weak entity as a *double box*).

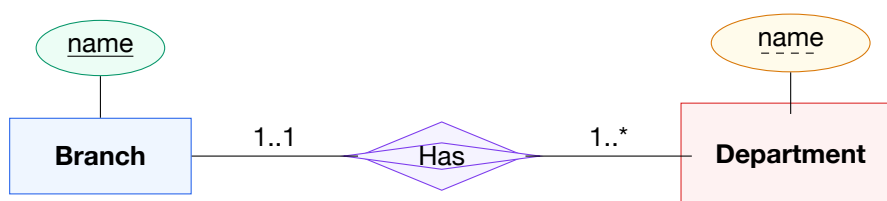
Its **discriminator (partial key)** is unique only *within* the owner.

Full key = discriminator + owner's primary key.

Department inside a Branch

Department names repeat across branches, but are unique within one branch. So Department is *weak*, identified by its Branch. Full key = (branchName, deptName).

Drawn out — note the **double box** (weak entity), **double diamond** (identifying relationship), and the *dashed underline* on the partial key:



(Reads: every Department belongs to exactly one Branch; a Branch has one or more Departments. “name” under Department is only a *partial* key.)

4.5 IS-A: Inheritance (Specialisation & Generalisation)

- **Specialisation** (top-down): split an entity set into subgroups.
- **Generalisation** (bottom-up): merge similar entity sets into a parent.
- Subclasses **inherit** all attributes and relationships of the parent.

- **Disjoint** ISA — entity in at most one subclass. **Overlapping** (default) — may be in several.
- **Total** specialisation — every parent entity is in at least one subclass.

4.6 Aggregation

Aggregation lets you treat a *whole relationship* as if it were an entity, so it can take part in *another* relationship. (Example: a `WorkAssignment` relationship that itself is-supervised-by an employee.)

4.7 ER vs. UML class diagrams

- UML puts **attributes inside the class box** (no ellipses).
- Binary relationships = plain lines; non-binary = a diamond.
- Keys: ER *underlines*; UML has no standard (people write PK).
- **Aggregation**: ER = “treat relationship as entity”; UML = “part-whole”.
- **Composition** (UML) $\approx$ weak entities, but says nothing about keys.

5 From Conceptual to Relational (Translation Rules)

This is a mechanical recipe. Memorise it — it shows up every exam.

ER → Relational tables

1. **Strong entity** → one table. Columns = attributes. PK = entity’s key.
2. **Weak entity** → table with its local attributes *plus* the owner’s PK (as part of the PK *and* an FK). The identifying relationship needs **no separate table**.
3. **Many-to-many** relationship → own table: the PKs of all participants (each an FK) + relationship attributes. PK = combination of participant keys.
4. **Many-to-one** (or 0..1 to 0..*) → **no new table**; put the “one” side’s PK as an FK column on the “many” side.
5. **One-to-one** → put one side’s key on the other; use `UNIQUE + NOT NULL`.
6. **Composite attribute** → flatten into one column per leaf (name → first, last).
7. **Multi-valued attribute** → own table: (owner PK as FK, value). One row per value.

Modelling cardinality with constraints

Total participation (1..1) $\Rightarrow$ FK `NOT NULL`. Partial (0..1) $\Rightarrow$ FK nullable. 1..1 to 1..1 $\Rightarrow$ **merge both entities into one table**.

Many-to-many turns into 3 tables

ER: Student – enrolled-in (grade) – Course, with 0..* on both sides. Because it is many-to-many *and* carries an attribute (grade), it needs its **own table**:

Student		Course		Enrolled (join table)		
<u>sid</u>	name	<u>cid</u>	title	<u>sid</u> →Stud	<u>cid</u> →Course	grade
1	Ana	DB	Databases	1	DB	8
2	Bob	OS	Operating Sys	1	OS	7
				2	DB	9

The join table’s PK is (sid, cid) — both columns together, each also an FK.

5.1 Eliminating tables

For many-to-(0..1 or 1..1) you can drop the relationship table and just add the “one” side’s key to the “many” side. Fewer tables = fewer joins = faster queries.

One-to-many: no extra table

ER: each Employee works in 1..1 Department; a department has 0..* employees. Instead of a separate “works-in” table, just put deptId as an FK column *on* Employee (NOT NULL, because participation is total):

Employee			Dept	
<u>id</u>	name	deptId → Dept (NOT NULL)	<u>id</u>	name
1	Ana	D1	D1	Sales
2	Bob	D1	D2	IT
3	Cleo	D2		

The “many” side (Employee) holds the FK. Putting it the other way round can’t work — a department would need many empId values in one cell, breaking 1NF.

5.2 ISA translation — 3 methods

Method	How	Trade-off
1. Hierarchy of tables	Superclass table + one table per subclass (local attrs + FK to super)	Needs joins
2. Many tables	One table per entity set, each with <i>all</i> inherited + local attrs	No joins; redundant for overlapping
3. One table + nulls	One wide table, all attributes; non-applicable = NULL	No joins; many nulls

5.3 Recursive relations

An entity related to itself (employee *supervises* employee). One-to-many → a self-referential FK in the same table. Many-to-many → a separate table with *two* FK columns pointing back to the same table.

Choosing a primary key: unique, *stable* (never updated), ideally single-column. Introduce a synthetic auto-increment `id` when natural keys are unstable, too wide, or not unique.

What the relational model *cannot* enforce

Plain SQL constraints can enforce: keys, NOT NULL, UNIQUE, foreign keys. They **cannot** enforce *minimum* cardinalities like “every branch has ≥ 1 department” or “every group has ≥ 10 children”. Those lower bounds need **triggers** / **assertions**. Always mention this in the “what cannot be modelled” exam part.

6 Functional Dependencies & Normalisation

Goal: kill **redundancy** so the same fact isn’t stored twice (which causes update bugs).

6.1 Why: the three anomalies

Anomalies from bad design

- **Insertion anomaly** — can't record a fact without unrelated data (can't add a department until an employee joins it).
- **Deletion anomaly** — deleting a row destroys another fact (last employee leaves $\Rightarrow$ the department disappears).
- **Update anomaly** — a duplicated value must change in many rows; miss one $\Rightarrow$ inconsistency (branch address copied on every employee row).

See the anomalies in one bad table

Enrol(sid, sName, advisor, advisorOffice, course) — advisor's office is repeated on every row for that advisor:

sid	sName	advisor	advisorOffice	course
1	Ana	Dr. Kim	R210	DB
1	Ana	Dr. Kim	R210	OS
2	Bob	Dr. Kim	R210	DB
3	Cleo	Dr. Lee	R305	DB

- **Update:** Dr. Kim moves office $\Rightarrow$ must change R210 in 3 rows; miss one $\Rightarrow$ inconsistent.
- **Insertion:** can't store a new advisor's office until they have a student + course.
- **Deletion:** delete Cleo's only row $\Rightarrow$ Dr. Lee's office R305 is gone.

Fix: the FD $\text{advisor} \rightarrow \text{advisorOffice}$ should live in its own table Advisor(advisor, advisorOffice).

6.2 First Normal Form (1NF)

All cell values are **atomic** — no lists, sets, or nested tables. The relational model already forces this. Every higher normal form assumes 1NF.

Not in 1NF vs. fixed

BAD — list in a cell		GOOD — one value per row	
sid	phones	sid	phone
1	{555-1, 555-2}	1	555-1
2	{555-9}	1	555-2
		2	555-9

6.3 Functional Dependencies (FDs)

FD definition

$\{A_1, \dots, A_n\} \rightarrow \{B_1, \dots, B_m\}$ holds in R if: *whenever two rows agree on all the A 's, they must also agree on all the B 's*. The left side **determines** the right side.

Keys are FDs: α is a key of R exactly when $\alpha \rightarrow \text{all attributes}$.

Reading an FD

In Student(sid, name, advisor, advisorOffice): $\text{sid} \rightarrow \text{name}$ (each student has one name), and $\text{advisor} \rightarrow \text{advisorOffice}$ (each advisor sits in one office). That second FD is the troublemaker — it repeats the office on every student of that advisor.

6.4 Armstrong's Axioms & Closure

Armstrong's axioms — sound & complete

- **Reflexivity:** if $\beta \subseteq \alpha$ then $\alpha \rightarrow \beta$ (trivial FDs).
- **Augmentation:** if $\alpha \rightarrow \beta$ then $\alpha\gamma \rightarrow \beta\gamma$.
- **Transitivity:** if $\alpha \rightarrow \beta$ and $\beta \rightarrow \gamma$ then $\alpha \rightarrow \gamma$.

Useful derived rules: **Union** ($\alpha \rightarrow \beta, \alpha \rightarrow \gamma \Rightarrow \alpha \rightarrow \beta\gamma$) and **Decomposition** (split a RHS).

Closure α^+ — the workhorse

The **closure** α^+ = all attributes that α determines, given FD set F .

1. Start: $\alpha^+ = \alpha$.
2. Repeat: if some FD $X \rightarrow Y$ has $X \subseteq \alpha^+$, add Y to α^+ .
3. Stop when nothing changes.

F implies $\alpha \rightarrow \beta$ iff $\beta \subseteq \alpha^+$.

Computing a closure

$F = \{C \rightarrow AB, BC \rightarrow D\}$. Find C^+ .

Start $C^+ = \{C\}$. $C \rightarrow AB$ adds $A, B \Rightarrow \{A, B, C\}$. Now $BC \rightarrow D$ fires (both in set) $\Rightarrow \{A, B, C, D\}$. Done. $C^+ = \{A, B, C, D\}$ — so C is a key if those are all attributes.

6.5 Canonical (Minimal) Set of FDs

Make FDs canonical

A **canonical** set: single attribute on each RHS, minimal LHS, no redundant FD.

1. **Split RHS** so every FD has one attribute on the right.
2. **Minimise LHS:** for $XY \rightarrow A$, if $A \in X^+$ (using all current FDs) then drop Y .
3. **Remove redundant FDs:** drop $X \rightarrow A$ if A is still in X^+ using the *remaining* FDs.

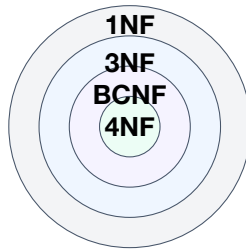
Order matters — the minimal cover is **not unique**.

6.6 Finding all minimal keys

Minimal keys from FDs

1. Any attribute appearing **only on left sides** (never on a RHS) is in *every* key — start there (the “core”).
2. Compute the core's closure. If it's everything, it's the only key.
3. Otherwise add the smallest groups of remaining attributes; a set is a **superkey** if its closure = all attributes, and a **key** if no proper subset is a superkey.

6.7 The Normal Forms



Stricter as you go inward.

$4NF \subset BCNF \subset 3NF \subset 1NF$.

A 4NF relation is automatically BCNF, which is automatically 3NF, etc.

The definitions — learn these word-for-word

BCNF: for every non-trivial FD $\alpha \rightarrow B$, the left side α **contains a key** (is a superkey).

3NF: for every non-trivial FD $\alpha \rightarrow B$: *either* α contains a key, *or* B is a **key attribute** (it appears in some minimal key).

4NF: for every non-trivial *multivalued* dependency $\alpha \twoheadrightarrow \beta$, α contains a key.

3NF is **weaker** (more forgiving) than BCNF — it adds the “or B is a key attribute” escape hatch. That extra slack is exactly what lets 3NF *always* preserve all FDs.

6.8 BCNF decomposition (always lossless, may lose FDs)

BCNF by decomposition

1. Find an FD $X \rightarrow Y$ that violates BCNF (X is not a superkey).
2. Split R into $S(X \cup Y)$ and R minus Y (keep X in both).
3. Repeat on each piece until all are in BCNF.

Always lossless, but **may not preserve all FDs**.

BCNF split, with data

$R(\text{sid}, \text{advisor}, \text{advisorOffice})$ with FD $\text{advisor} \rightarrow \text{advisorOffice}$. The key is sid , but advisor is *not* a superkey $\Rightarrow$ that FD violates BCNF. Split on it:

$\Rightarrow$

R (before — redundant)			Studies		Adv	
sid	advisor	advisorOffice	sid	advisor $\rightarrow$ Adv	advisor	advisorOffice
1	Dr. Kim	R210	1	Dr. Kim	Dr. Kim	R210
2	Dr. Kim	R210	2	Dr. Kim	Dr. Lee	R305
3	Dr. Lee	R305	3	Dr. Lee		

Office now stored **once** per advisor. Join back on `advisor` to recover the original — that is what *lossless* means.

6.9 3NF synthesis (always lossless and FD-preserving)

3NF synthesis algorithm

1. Compute a **canonical** (minimal) FD set F .
2. **Merge** FDs with the same LHS into one relation each: $X \rightarrow Y_1 \dots Y_k$ becomes $R_i(X, Y_1, \dots, Y_k)$, key X .
3. If *no* relation contains a key of the original R , add one relation that *is* a minimal key.
4. Drop any relation whose attributes are a subset of another's.

3NF synthesis — quick run

$R(A, B, C, D, E)$, canonical $F = \{A \rightarrow B, A \rightarrow C, BCD \rightarrow E, E \rightarrow B, E \rightarrow C, AE \rightarrow D\}$.
 Group by LHS: $A \rightarrow BC \Rightarrow R_1(A, B, C)$; $BCD \rightarrow E \Rightarrow R_2(B, C, D, E)$; $E \rightarrow BC \Rightarrow R_3(E, B, C)$; $AE \rightarrow D \Rightarrow R_4(A, E, D)$.
 A, D never appear on a RHS $\Rightarrow$ in every key; $AD^+ = \text{all}$, so $\{A, D\}$ is a key. $R_4(A, E, D)$ contains A, D ? No, missing nothing needed — but check: a relation must contain a full key $\{A, D\}$. R_4 has A, D ✓, so no extra key relation needed.

6.10 Multivalued Dependencies (MVDs) & 4NF

MVD

$A \twoheadrightarrow B$ holds if, for two rows agreeing on A , swapping their B values still gives valid rows. MVDs come in **pairs**: $A \twoheadrightarrow B$ and $A \twoheadrightarrow \text{rest}$. **4NF** requires every non-trivial MVD's LHS to be a superkey.

6.11 Denormalisation

Sometimes you *deliberately* break a normal form, storing redundant data to make reads faster. Trade-off: **faster reads** vs. **costlier updates + risk of inconsistency**.

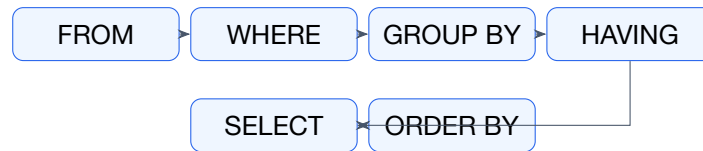
Form	Condition (every non-trivial FD/MVD)	FD-preserving?
1NF	All values atomic	Yes
3NF	LHS is superkey or RHS is a key attribute	Always (synthesis)
BCNF	LHS contains a superkey	Not always
4NF	LHS of every MVD contains a superkey	Not always

7 SQL — Structured Query Language

7.1 Sample data used in the SQL examples

Person				Knows (A knows B)		Friend (symmetric)	
id	name	city	age	id1	id2	id1	id2
1	Ana	Paris	35	1	2	1	2
2	Bob	Paris	28	1	3	2	1
3	Cleo	Rome	52	2	3	3	4
4	Dan	Rome	19	4	1	4	3

7.2 The shape of a query & its logical order



Required clauses + how to picture them

- **SELECT** — which output columns (* = all).
- **FROM** — the tables (tuple variables); listing several = **Cartesian product**.
- **WHERE** — row filter (no filter $\equiv$ TRUE).

Mental model: nested for-each loops over all row combinations in **FROM**, keep those passing **WHERE**, output the **SELECT** columns.

7.3 Joins

List tables in **FROM**, then put the matching condition in **WHERE** (link key = FK).

Always connect every table

If a tuple variable isn't linked to the others, you get a meaningless cross-product (every row paired with every row). Always join *all* variables, directly or transitively.

Join

```
SELECT e.employeeName, c.city
FROM Works w, Employee e, Company c
WHERE w.employeeName = e.employeeName
      AND w.companyName = c.companyName
      AND w.companyName = 'First Bank Corporation';
```

7.4 Self joins

Use **two aliases for the same table** to compare a row with other rows of that table. Add an inequality so you don't compare a row to itself.

7.5 Duplicates

SQL **keeps duplicates** by default. **DISTINCT** removes them.

Don't paper over bugs with DISTINCT

Surprise duplicates usually mean a *missing join condition* — fix the query, don't slap on **DISTINCT**. (**DISTINCT** is also pointless and slow when your **SELECT** already contains a key.)

7.6 Non-monotonic queries (negation): the hard ones

"none / not / for all / min / max" need subqueries

These can't be done with plain joins:

- `NOT IN` — value absent from a subquery result.
- `NOT EXISTS` — a (correlated) subquery returns zero rows.
- `EXISTS` — subquery returns *at least one* row.
- `ALL / ANY / SOME` — compare a value against all/some of a column.

"For all" trick: $\forall x \varphi \equiv \neg \exists x \neg \varphi$. Translate "X relates to every Y" into **double NOT EXISTS** ("there is no Y that X is *not* related to") — this is **relational division**.

Relational division — "knows everybody in Paris"

```
SELECT p.id
FROM Persons p
WHERE NOT EXISTS ( -- no Parisian...
  SELECT 1 FROM Persons par
  WHERE par.address = 'Paris'
  AND NOT EXISTS ( -- ...whom p does NOT know
    SELECT 1 FROM Knows k
    WHERE k.personA_id = p.id
    AND k.personB_id = par.id));
```

7.7 Subqueries — where they go

- **In WHERE** — a set for `IN/NOT IN/EXISTS/comparison`.
- **Scalar subquery** — exactly one column, at most one row; used in comparisons.
- **In FROM** — a derived table (must be aliased); lets you nest aggregations.
- **In SELECT** — correlated scalar subquery, one value per output row.
- **Views** — named subqueries stored in the DB, used like tables.

7.8 Aggregation

Aggregate functions collapse many values to one

`COUNT(*)` counts rows (incl. nulls); `COUNT(A)` counts non-null *A*; `SUM/AVG` over non-nulls; `MIN/MAX` ignore nulls. Use `COUNT(DISTINCT A)` to dedupe. **No aggregates in WHERE**; can't nest aggregates directly (use a subquery in FROM).

7.9 GROUP BY and HAVING

Group, then filter groups

`GROUP BY` partitions rows by equal values; aggregates apply *per group*. Only `GROUP BY` columns + aggregates may appear in `SELECT`.

`WHERE` filters rows *before grouping*; `HAVING` filters whole groups *after aggregation*.

WHERE vs HAVING

```
SELECT l.personA_id AS id
FROM Likes l JOIN Persons p ON l.personB_id = p.id
GROUP BY l.personA_id
HAVING AVG(p.age) BETWEEN 30 AND 60; -- a group condition
```

Trace a GROUP BY on the sample data

“How many people does each person know?”

```
SELECT id1, COUNT(*) AS known
FROM Knows GROUP BY id1;
```

Group the Knows rows by id1, then count each group:

groups of Knows		⇒		result	
id1	rows			id1	known
1	(1,2),(1,3)			1	2
2	(2,3)			2	1
4	(4,1)			4	1

Add HAVING COUNT(*) >= 2 and only person 1 survives — HAVING filters whole groups *after* counting.

7.10 Set operations

- UNION (removes dups) / UNION ALL (keeps them).
- EXCEPT ($A - B$): rows in A not in B .
- INTERSECT ($A \cap B$): rows in both.
- Operands need the **same number of columns**, compatible types, matched by *position*.

7.11 CASE and COALESCE

CASE WHEN cond THEN val ... ELSE default END for conditional values in SELECT (cleaner/faster than equivalent UNIONS). COALESCE(a,b,...) returns the first non-null.

8 Transactions

A **transaction** = a unit of work that must keep **ACID** even with crashes and many users at once.

8.1 What goes wrong without isolation (anomalies)**The four concurrency anomalies**

- **Lost Update** — T_1 and T_2 both read X , compute, write back; the second write overwrites the first. T_1 's update vanishes.
- **Dirty Read** — T_2 reads X that T_1 wrote but *hasn't committed*. If T_1 aborts, T_2 used data that “never happened”.
- **Inconsistent Read** — T_1 reads several tables at different moments while T_2 updates them; T_1 sees a half-updated state.
- **Unrepeatable Read** — T_1 reads X , T_2 updates & commits, T_1 re-reads X and gets a different value.

8.2 Formal model: schedules & serializability

Definitions

A **transaction** = a sequence of $\text{Read}(X)$ / $\text{Write}(X)$ ending in **Commit** or **Abort**.

A **schedule** = an interleaving of actions from several transactions, keeping each transaction's own order.

A **serial schedule** runs transactions one after another (no interleaving).

A schedule is **serializable** if its effect equals *some* serial schedule.

8.3 Conflicts & conflict serializability

When two actions conflict

Two actions conflict if: **(1)** different transactions, **(2)** same data item, **(3)** at least one is a **Write**.

- **WR** (T_1 writes, T_2 reads): possible dirty read.
- **RW** (T_1 reads, T_2 writes): T_1 's read becomes unrepeatable.
- **WW** (T_1 writes, T_2 writes): first write overwritten.

Two schedules are **conflict equivalent** if you can turn one into the other by swapping adjacent *non-conflicting* actions. A schedule is **conflict serializable** if it's conflict equivalent to some serial schedule.

8.4 Precedence graph — the fast test

Is it conflict serializable?

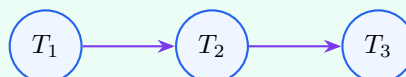
1. One **node** per transaction.
2. For each conflicting pair where T_i 's action comes *before* T_j 's, add edge $T_i \rightarrow T_j$.
3. **Conflict serializable** $\iff$ **the graph has no cycle**. A **topological sort** gives an equivalent serial order.

Worked precedence graph

Schedule shown on a timeline (each transaction has its own row; read left to right):

	time $\longrightarrow$	
T_1	$R_1(x)$	$W_1(y)$
T_2	$W_2(x)$	$R_2(y)$
T_3	$R_3(x)$	

Find conflicts (same item, different txn, ≥ 1 write): $R_1(x)$ before $W_2(x)$ (RW) $\Rightarrow T_1 \rightarrow T_2$; $W_2(x)$ before $R_3(x)$ (WR) $\Rightarrow T_2 \rightarrow T_3$; $W_1(y)$ before $R_2(y)$ (WR) $\Rightarrow T_1 \rightarrow T_2$ (already). Draw it:



No cycle $\Rightarrow$ conflict serializable. Topological sort = serial order T_1, T_2, T_3 .

A cycle = NOT serializable

If the schedule were $R_1(x), R_2(x), W_1(x), W_2(x)$ you would get *both* $T_1 \rightarrow T_2$ and $T_2 \rightarrow T_1$:



A 2-node cycle $\Rightarrow$ no serial order exists $\Rightarrow$ **not** conflict serializable.

8.5 Two-Phase Locking (2PL)

2PL guarantees conflict serializability at runtime

- Take a **shared (S)** lock before reading, an **exclusive (X)** lock before writing.
- Many S-locks on one item are fine together; an X-lock needs exclusive access.
- **Phase 1 (Growing)**: only *acquire* locks. **Phase 2 (Shrinking)**: once you *release* any lock, you may never acquire another.

Deadlock

T_1 waits for T_2 's lock while T_2 waits for T_1 's. Detect with a **wait-for graph** (cycle = deadlock); resolve by **aborting** one transaction.

8.6 Flavours of 2PL (exam favourites)

Variant	Extra rule
Basic 2PL	Grow then shrink. Prevents non-serializable schedules.
Strict 2PL	Hold <i>all X-locks until commit/abort</i> . Prevents dirty reads $\Rightarrow$ cascadeless & recoverable. Most common in practice.
Rigorous 2PL	Hold <i>all</i> locks (S and X) until commit.
Preclaiming (conservative) 2PL	Acquire <i>all</i> locks <i>before starting</i> . No deadlocks, but you must know your locks up front.

8.7 Recoverable & cascadeless schedules

Two safety properties

Recoverable: a transaction commits only *after* every transaction whose data it read has already committed. (Stops you committing on top of data that later disappears.)

Cascadeless (avoids cascading aborts): a transaction may only read data written by *already-committed* transactions. So aborting one transaction never forces others to abort.

Cascadeless $\Rightarrow$ recoverable. Strict 2PL gives both.

Cascading rollback

If T_2 read a value T_4 wrote, and T_4 is now **aborted**, then T_2 must roll back too (and anything that read from T_2 , and so on). That chain is a **cascading rollback**.

8.8 Other concurrency control

- **Timestamp-based**: give each transaction a timestamp; reject any operation that breaks the timestamp (serial) order.

- **Optimistic (OCC):** run freely while tracking a **read-set** and **write-set**; *validate* at commit; abort if a conflict is found. (*Backward* OCC checks against already-committed transactions.)
- **MVCC (multi-version):** keep several versions; read-only queries see a consistent *snapshot* with no locking.

8.9 Multi-granularity locking (intention locks)

You can lock at table, page, or row level. **Intention locks** (IS, IX) on a big object signal that a finer lock sits below. Compatibility matrix (✓ = OK together, ✗ = conflict):

	IS	IX	S	X
IS	✓	✓	✓	✗
IX	✓	✓	✗	✗
S	✓	✗	✓	✗
X	✗	✗	✗	✗

Memory hook: **X conflicts with everything**; the intention locks (IS/IX) are friendly with each other and only clash where a real read/write would.

9 Database Application Programming

9.1 Application architectures

- **Thin-client** — all logic in the DB server; client just displays.
- **Client-server** — an app server holds business logic; DB server handles data.
- **Embedded** — the DB lives inside the app process (e.g. SQLite).

9.2 Dynamic SQL with JDBC

The JDBC pattern

Connection → Statement → `executeQuery()` → ResultSet → read rows → close. SQL types map to Java types (`varchar`→`String`, `integer`→`int`); the match isn't always exact, so call `rs.isNull()` after reading possibly-null values.

- **Prepared statements** — pre-compile a query with ? placeholders; pass values via the API. Lets the DBMS optimise repeated queries.
- **Connection pooling** — reuse open connections to skip setup cost.
- **Stored procedures** — DB-side logic called via `Connection.prepareCall()`.

9.3 SQL Injection — the one security rule

NEVER concatenate user input into SQL

Building SQL by gluing strings lets input *change the query structure*.

```
"SELECT * FROM Users WHERE name = '" + input + '"
```

With `input = ' OR '1'='1`, the WHERE becomes always-true and returns everyone. `'`; `DROP TABLE Users`; `--` can delete data. **Always use PreparedStatement with ? placeholders** so values are bound separately and can never become code.

Other defences: ORM/query builders (parameterise by default), input allow-lists for things you can't parameterise (table/column names), escaping (weakest), least-privilege DB accounts.

9.4 Object-Relational Mapping (ORM)

ORM hides SQL behind objects

Table $\approx$ class, row $\approx$ object, FK navigation $\approx$ following an object reference. Examples: JPA/Hibernate (Java), Entity Framework (.NET), Rails ActiveRecord. You usually write an **object query language** (HQL, LINQ) instead of raw SQL.

ORM dangers

N+1 query problem: looping over objects and touching a related field inside the loop fires one SQL query per iteration instead of one batch query. Also: *performance opacity* (hidden generated SQL is hard to debug). **Fix:** use HQL/LINQ with explicit joins, not object navigation in loops.

9.5 LINQ

LINQ bakes query syntax into C#/VB as real language constructs (not strings): full **compile-time type checking**, works on any `IEnumerable` (lists, XML, databases), and is translated to SQL at run-time. Downside: **not portable** — .NET only (similar tools elsewhere: SQLAlchemy, Django ORM).

Impedance mismatch: the relational model (sets of tuples) doesn't map cleanly onto the object model (objects with references) — that gap is why ORMs exist (and why they leak). Relate ORMs to ANSI-SPARC: they add an extra *view* layer on top of the logical schema.

10 One-Page Cheat Sheet

Vocabulary

- **Relation** = set of tuples; no dups, no order.
- **Primary key** = minimal unique identifier, never null.
- **Foreign key** = column referencing another table's key.
- **FD** $\alpha \rightarrow \beta$ = equal α forces equal β .
- **Closure** α^+ = all attrs α determines.
- **Superkey** = closure is everything.

Normal forms

- 1NF: atomic cells.
- 3NF: LHS superkey OR RHS key-attribute.
- BCNF: LHS always a superkey.
- 4NF: same for MVDs.
- 3NF synthesis: lossless + FD-preserving.
- BCNF decomp: lossless, maybe not FD-preserving.

SQL order FROM $\rightarrow$ WHERE $\rightarrow$ GROUP BY $\rightarrow$ HAVING $\rightarrow$ SELECT $\rightarrow$ ORDER BY.
WHERE = rows; HAVING = groups.
“for all” = double NOT EXISTS.

Transactions

- ACID = Atomic, Consistent, Isolated, Durable.
- Conflict = same item, diff txn, ≥ 1 write.
- Serializable $\iff$ precedence graph acyclic.
- 2PL: grow then shrink.
- Strict 2PL: hold X-locks to commit $\Rightarrow$ cascadeless.

Programming

- Always use `PreparedStatement` (anti-injection).
- Watch the ORM N+1 problem.
- ORM = extra view layer (ANSI-SPARC).

11 Exercises

Work these on paper first, then check the **Answer Key** that follows. Schema for the SQL set:

```
Person(id, name, city, age)
Knows(id1 -> Person, id2 -> Person) -- A knows B (not symmetric)
Friend(id1 -> Person, id2 -> Person) -- symmetric
Account(accountNr, balance)
BelongsTo(id -> Person, accountNr -> Account)
```

A. Relational Model & Modelling

- A1.** Define *primary key*, *candidate key*, and *superkey*, and say how they relate.
- A2.** Why does `WHERE phone = NULL` return nothing? What should you write instead?
- A3.** A Department name is unique only inside its Branch. Is Department a strong or weak entity? What is its full key, and how is it translated to a table?
- A4.** Give the three ISA translation methods and one trade-off of each.
- A5.** In an ER model, how do you draw: a multi-valued attribute, a derived attribute, a weak entity, an identifying relationship?

B. Functional Dependencies & Normalisation

- B1.** $R(A, B, C, D, E)$ with $F = \{A \rightarrow B, A \rightarrow C, BC \rightarrow D, C \rightarrow E\}$. Compute A^+ and C^+ . Is A a key?
- B2.** For the same R and F , find all minimal keys.
- B3.** $R(A, B, C, D, E)$ with $D \rightarrow A, BC \rightarrow D, ABC \rightarrow DE, C \rightarrow D, BE \rightarrow AC$. Compute a **canonical** (minimal) set of FDs.
- B4.** $R(A, B, C, D, E)$, canonical $F = \{BCD \rightarrow E, A \rightarrow B, E \rightarrow B, E \rightarrow C, AE \rightarrow D, A \rightarrow C\}$. Use **synthesis** to give a **BCNF** (here also 3NF) decomposition. Which FDs, if any, are lost?
- B5.** Name the three anomalies and give a one-line example of each.

C. SQL (use the Person schema above)

- C1.** Names of all persons younger than 40.
- C2.** Names of all persons that have a friend — using `EXISTS`. Will duplicates appear?
- C3.** Names of all persons that have *no* friends — once with `NOT IN`, once with `NOT EXISTS`.
- C4.** Names of persons with a bank account holding more than 1000 euros (no duplicate names).
- C5.** Persons that know at least two people — (a) without `GROUP BY`, (b) with `GROUP BY`.
- C6.** (*Suppliers/Parts*) `Catalog(sid→Suppliers, pid→Parts, cost)`. Find the pids of parts supplied by *only one* supplier — **without** `GROUP BY`.

D. Transactions

- D1.** Schedule $S = R_1(x), R_2(x), W_1(x), W_2(x)$. Draw the precedence graph. Is it conflict serializable?
- D2.** Schedule $R_1(x), W_1(x), R_2(x)$: insert lock/unlock actions so it obeys 2PL.
- D3.** Define *recoverable* and *cascadeless* schedules. Which does strict 2PL guarantee?
- D4.** For multi-granularity locks, say TRUE/FALSE (conflict?): (i) X-X, (ii) S-S, (iii) S-X, (iv) IS-X, (v) IX-S, (vi) IX-IX.

E. Application Programming

- E1.** Show how the input ' OR '1'='1 breaks a string-concatenated login query, and give the fix.
- E2.** What is the ORM N+1 query problem and how do you avoid it?
- E3.** Name the three ANSI-SPARC levels. At which level do ORMs like Hibernate sit, and why?

12 Answer Key

A. Relational Model & Modelling

- A1.** A **superkey** is any set of attributes that is unique per row. A **candidate (minimal) key** is a superkey with no removable attribute. The **primary key** is the one candidate key you choose as the main identifier (never null, never updated); the rest are alternate keys. So: primary key $\subseteq$ candidate keys $\subseteq$ superkeys.
- A2.** Because any comparison with NULL yields UNKNOWN (three-valued logic), and WHERE only keeps TRUE rows. Use WHERE phone IS NULL.
- A3.** **Weak** entity (no key of its own). Full key = its discriminator + the owner's key, i.e. (branchName, deptName). Translation: a table Department(branchName, name, ...) where branchName is both part of the PK and an FK to Branch; the identifying relationship needs no separate table.
- A4.** (1) *Hierarchy of tables*: super + subclass tables, joins needed. (2) *Many tables*: one full table per entity set, no joins but redundant for overlapping ISA. (3) *One table + nulls*: single wide table, no joins but many nulls.
- A5.** Multi-valued = *double* ellipse; derived = *dashed* ellipse; weak entity = *double* rectangle; identifying relationship = *double* diamond.

B. Functional Dependencies & Normalisation

- B1.** A^+ : start $\{A\}$; $A \rightarrow B, A \rightarrow C$ add $B, C \Rightarrow \{A, B, C\}$; $C \rightarrow E$ adds E ; $BC \rightarrow D$ adds $D \Rightarrow \{A, B, C, D, E\} = \text{all}$. So **A is a key**.
 C^+ : $\{C\}$; $C \rightarrow E \Rightarrow \{C, E\}$. Nothing else fires. $C^+ = \{C, E\}$ — not a key.
- B2.** A appears only on left sides $\Rightarrow A$ is in every key. $A^+ = \text{all}$, so **$\{A\}$ is the only minimal key**.
- B3.** Singletons + remove extraneous/redundant. One valid minimal cover: $C \rightarrow D, D \rightarrow A, BC \rightarrow E, BE \rightarrow C$ (and A is determined via $C \rightarrow D \rightarrow A$). A clean accepted answer: $\{B, C \rightarrow D, D \rightarrow A, A, B, C \rightarrow E, B, E \rightarrow A, C\}$ reduced to $\{C \rightarrow D, D \rightarrow A, BC \rightarrow E, BE \rightarrow C\}$. (Minimal covers are not unique; check yours by verifying each side's closure matches the original.)
- B4.** Group by LHS: $A \rightarrow BC \Rightarrow R_1(\underline{A}, B, C)$; $BCD \rightarrow E \Rightarrow R_2(\underline{B}, \underline{C}, D, E)$; $E \rightarrow BC \Rightarrow R_3(\underline{E}, B, C)$; $AE \rightarrow D \Rightarrow R_4(A, \underline{E}, D)$. A, D never appear on a RHS $\Rightarrow$ in every key; $AD^+ = \text{all}$ so $\{A, D\}$ is a key — and R_4 already contains $\{A, D\}$, so no extra key relation is needed. **No FD is lost** (synthesis is FD-preserving). (If a question forces a pure BCNF decomposition instead, an FD like $E \rightarrow B$ can become unenforceable in a single table — that is the usual “lost FD”.)
- B5.** **Insertion**: can't add a department before an employee joins it. **Deletion**: removing the last employee of a department erases the department too. **Update**: a branch address copied on every employee row must be changed everywhere at once.

C. SQL

C1.

```
SELECT name FROM Person WHERE age < 40;
```

```
SELECT p.name FROM Person p
WHERE EXISTS (SELECT 1 FROM Friend f WHERE f.id1 = p.id);
```

No duplicates: EXISTS matches a person *once* regardless of how many friends. (A plain join on Friend *would* duplicate; that's why EXISTS or DISTINCT is needed.)

C2.

```
-- NOT IN
SELECT name FROM Person
WHERE id NOT IN (SELECT id1 FROM Friend);
```

```
-- NOT EXISTS
SELECT p.name FROM Person p
WHERE NOT EXISTS (SELECT 1 FROM Friend f WHERE f.id1 = p.id);
```

```
SELECT DISTINCT p.name
FROM Person p
JOIN BelongsTo b ON b.id = p.id
JOIN Account a ON a.accountNr = b.accountNr
WHERE a.balance > 1000;
```

(DISTINCT because a person may own several qualifying accounts.)

C5.

```
-- (a) without GROUP BY: self-join two distinct known persons
SELECT DISTINCT k1.id1
FROM Knows k1, Knows k2
WHERE k1.id1 = k2.id1 AND k1.id2 <> k2.id2;

-- (b) with GROUP BY
SELECT id1
FROM Knows
GROUP BY id1
HAVING COUNT(DISTINCT id2) >= 2;
```

```
SELECT DISTINCT c1.pid
FROM Catalog c1
WHERE NOT EXISTS (
    SELECT 1 FROM Catalog c2
    WHERE c2.pid = c1.pid AND c2.sid <> c1.sid);
```

“No *other* supplier sells this part” = supplied by exactly one.

D. Transactions

D6. Conflicts: $R_1(x)$ before $W_2(x)$ (RW) $\Rightarrow T_1 \rightarrow T_2$; $R_2(x)$ before $W_1(x)$ (RW) $\Rightarrow T_2 \rightarrow T_1$; ($W_1(x)$ before $W_2(x)$ also $T_1 \rightarrow T_2$). Edges $T_1 \rightarrow T_2$ and $T_2 \rightarrow T_1$ form a **cycle** $\Rightarrow$ **not conflict serializable**.

D2.

```
T1: lock-X(x); R1(x); W1(x); unlock(x) -- all locks before any unlock
T2: lock-X(x); R2(x); unlock(x)
```

T_2 's lock-X(x) waits until T_1 releases x . Each transaction takes all its locks before releasing any $\Rightarrow$ obeys 2PL.

D3. Recoverable: a transaction commits only after every transaction it read from has committed. **Cascadeless:** a transaction only reads data written by already-committed transactions (so no abort cascades). **Strict 2PL** guarantees *both* (cascadeless $\Rightarrow$ recoverable).

D4. (i) X-X = **TRUE** (conflict). (ii) S-S = **FALSE**. (iii) S-X = **TRUE**. (iv) IS-X = **TRUE**. (v) IX-S = **TRUE**. (vi) IX-IX = **FALSE**.

E. Application Programming

E1. Query "... WHERE name = '" + input + "'" becomes ... WHERE name = '' OR '1'='1', which is always true and returns/authorises every row. Fix:

```
PreparedStatement ps =
    conn.prepareStatement("SELECT * FROM Users WHERE name = ?");
ps.setString(1, input); -- value bound separately, never code
```

- E2. N+1:** a first query loads N objects, then touching a related field inside a loop fires one more query per object — $1 + N$ queries instead of one. Avoid with an explicit join / eager fetch in HQL/LINQ, or a single batch query.
- E3. External (view), Logical (conceptual), Internal (physical).** ORM's sit at the **highest / external (view)** level: they give the application its own object-shaped view on top of the logical schema, bridging the impedance mismatch between objects and relations.

Good luck on the exam!

Skim the cheat sheet one last time, redo any exercise you missed, and remember: closures for keys, precedence graphs for serializability, double-NOT-EXISTS for “for all”, and *always* PreparedStatement.